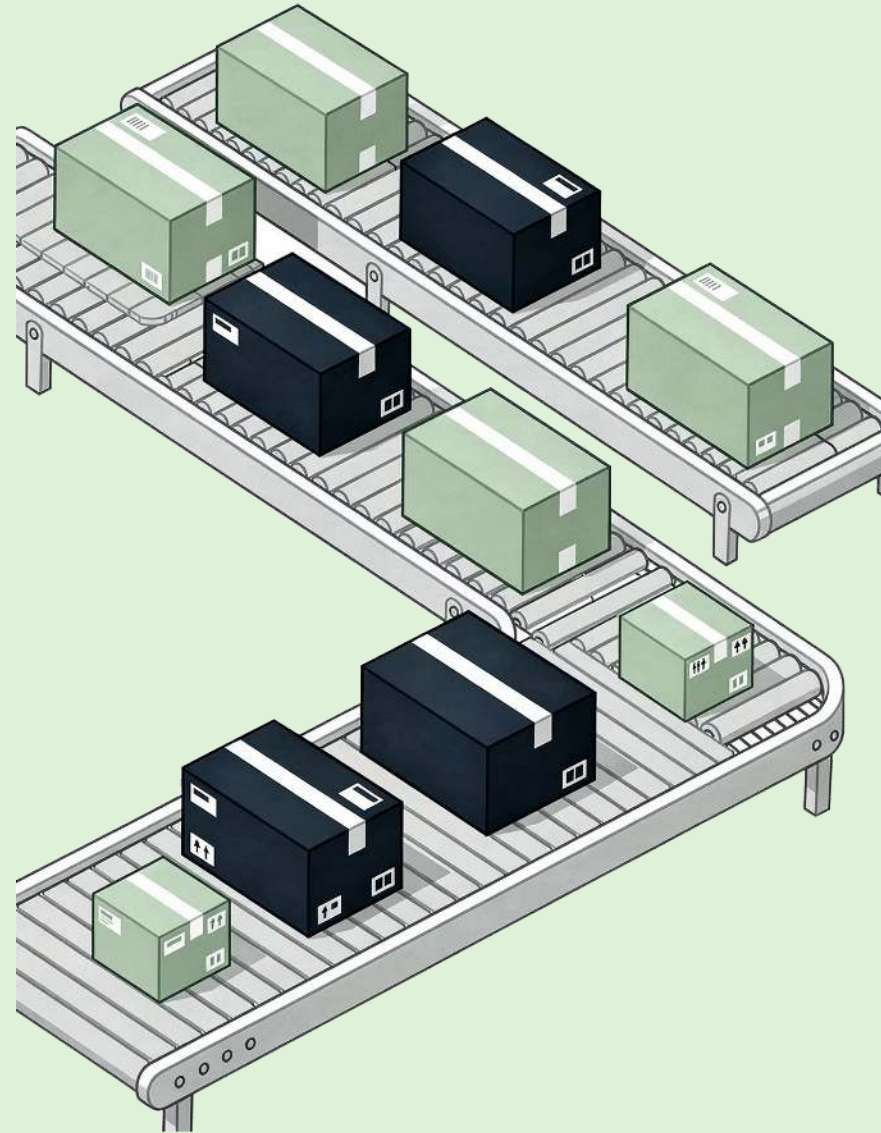


Mini-Shop als Microservice-Architektur

SOFTWARE ENGINEERING BEISPIELPROJEKT

APIs, Docker, Docker Compose, Kubernetes und CI/CD anhand eines kleinen Online-Shops – von einfachen lokalen Prozessen bis zur automatisierten Deployment-Pipeline.



Ziel des Projekts



Microservices verstehen

Microservices praktisch erleben und die Konzepte dahinter nachvollziehen



HTTP-Kommunikation

Kommunikation zwischen Services über HTTP-APIs nachvollziehen



Containerisierung

Services mit Docker in isolierte Container verpacken und betreiben



Orchestrierung

Compose und Kubernetes vergleichen und deren Unterschiede verstehen



CI/CD

Grundidee von Continuous Integration mit GitHub Actions demonstrieren

Fachliche Domäne

Ein minimaler Online-Shop – bewusst klein gehalten, damit **Infrastruktur und Kommunikation** im Vordergrund stehen.



Produkte anzeigen

Produktliste abrufen und einzelne Produkte anzeigen



Lagerbestand prüfen

Verfügbarkeit von Produkten im Lager abfragen



Bestellung anlegen

Neue Bestellungen erstellen und verarbeiten



Services

Der Mini-Shop besteht aus vier klar getrennten Services, die jeweils auf einem eigenen Port laufen:

1

api-gateway

Einstiegspunkt für Clients – Port 3000

2

product-service

Produktdaten verwalten – Port 3001

3

inventory-service

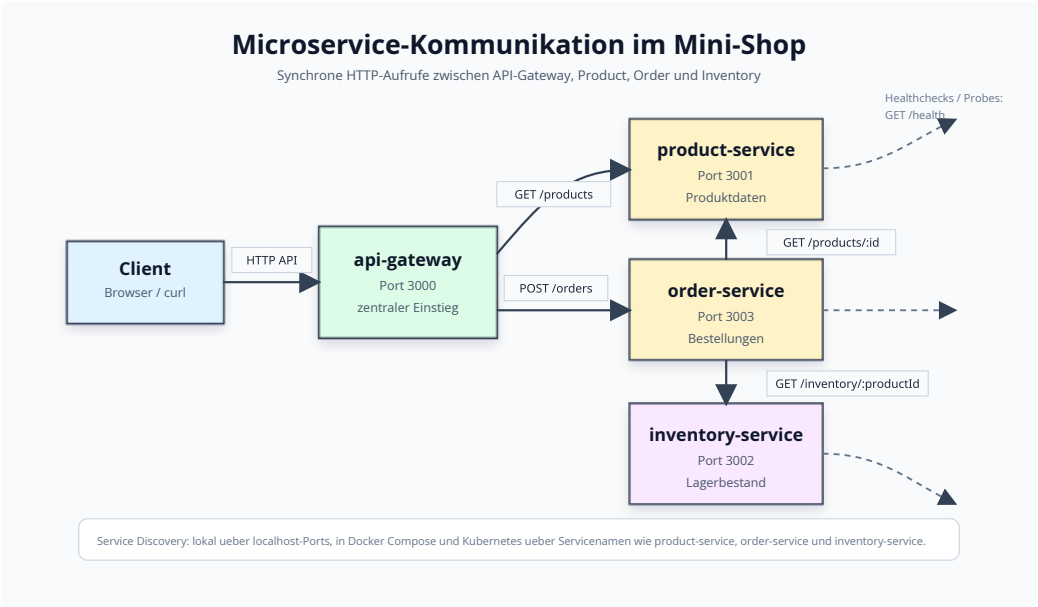
Lagerbestand verwalten – Port 3002

4

order-service

Bestellungen verarbeiten – Port 3003

Kommunikation





Warum Microservices?

Vorteile

- Kleine, fachlich geschnittene Komponenten
- Services können getrennt entwickelt und betrieben werden
- Klare APIs zwischen Komponenten
- Technische Abhängigkeiten werden sichtbar

Tradeoffs

Verteilte Systeme bringen neue Herausforderungen mit sich:

- Netzwerkfehler müssen behandelt werden
- Erhöhter Deployment-Aufwand
- Monitoring-Bedarf steigt

⚠ Microservices sind kein Allheilmittel – der Mehraufwand muss bewusst in Kauf genommen werden.

ITERATION 1

Lokal ohne Docker

In der ersten Iteration werden alle Services als eigenständige Prozesse direkt auf dem lokalen Rechner gestartet – ohne Container.



APIs verstehen

HTTP-APIs und Service-Kommunikation direkt beobachten und nachvollziehen



Prozesse starten

Jeden Service als eigenen Prozess auf seinem Port starten: 3000, 3001, 3002, 3003



Fehler beobachten

Erste Fehlerfälle erleben, wenn ein Downstream-Service nicht erreichbar ist

Lernpunkte: Lokale Services

Kernfragen

- Was ist ein Service?
- Was ist eine HTTP-API?
- Wie ruft ein Service einen anderen Service auf?
- Was passiert, wenn ein Downstream-Service nicht erreichbar ist?

Beispielaufruf

```
curl http://localhost:3000/products
```

i Dieser einfache Aufruf durchläuft das API-Gateway und ruft intern den product-service auf Port 3001 auf.

Dockerfiles

In der zweiten Iteration wird jeder Service in ein eigenes Docker-Image verpackt. Das Prinzip: **ein Verantwortungsbereich pro Container**.

package.json

Abhängigkeiten und Skripte des Services

src/index.js

Hauptdatei des Service-Codes

Dockerfile

Bauanleitung für das Docker-Image

.dockerignore

Ausschluss unnötiger Dateien aus dem Image



Lernpunkte: Dockerfiles

Wichtige Dockerfile-Befehle

FROM node:22-alpine

Basis-Image definieren

WORKDIR

Arbeitsverzeichnis im Container setzen

COPY

Dateien in den Container kopieren

RUN npm ci --omit=dev

Abhängigkeiten reproduzierbar installieren

EXPOSE

Port dokumentieren

CMD

Startbefehl des Containers

Netzwerkcommunication

Im Docker-Netzwerk erreichen sich Services über ihre **Container-Namen** statt über localhost.

i Statt `localhost:3001` wird `product-service:3001` verwendet – der Container-Name fungiert als DNS-Eintrag.



ITERATION 3

Docker Compose

In der dritten Iteration werden alle Services gemeinsam über eine zentrale Konfigurationsdatei gestartet – manuelle `docker run`-Befehle gehören der Vergangenheit an.

Gemeinsamer Start

Alle Services mit einem einzigen Befehl starten

Zentrale Konfiguration

Service-Konfiguration in einer einzigen YAML-Datei beschreiben

Automatisches Netzwerk

Compose erstellt automatisch ein gemeinsames Netzwerk für alle Services

```
docker compose -f compose/docker-compose.yml up --build
```

Lernpunkte: Docker Compose

Kernkonzepte



Automatisches Netzwerk

Compose erstellt automatisch ein Netzwerk – Services erreichen sich über Servicenamen



Environment Variables

Abhängigkeiten werden über Umgebungsvariablen konfiguriert



depends_on

Beschreibt die Startreihenfolge der Services

Beispielkonfiguration

```
PRODUCT_SERVICE_URL=http://product-service:3001
```

- ✓ Mit Compose wird die gesamte lokale Infrastruktur als Code beschrieben und ist damit reproduzierbar und versionierbar.



ITERATION 4

Healthchecks

In der vierten Iteration wird die **Betriebsbereitschaft der Services sichtbar gemacht**. Der entscheidende Unterschied: Ein laufender Container bedeutet nicht automatisch einen betriebsbereiten Service.

Container läuft

Der Docker-Prozess ist gestartet und der Container ist aktiv

Service ist bereit

Der Service hat erfolgreich gestartet und beantwortet Anfragen

Jeder Service stellt einen Health-Endpunkt bereit: `GET /health` – Compose prüft diesen Endpunkt regelmäßig.

Lernpunkte: Healthchecks



Verbesserte Orchestrierung

Healthchecks verbessern die lokale Orchestrierung erheblich – Services starten erst, wenn ihre Abhängigkeiten wirklich bereit sind



Warten auf Bereitschaft

`depends_on: condition:`
`service_healthy` wartet auf bereite Services statt nur auf gestartete Container



Frühere Fehlererkennung

Fehler werden früher sichtbar, bevor sie sich als kaskadierendes Problem manifestieren

 **Wichtig:** Compose-Healthchecks sind Docker-Compose-spezifisch und werden nicht automatisch von Kubernetes verwendet.

Kubernetes

In der fünften Iteration werden die containerisierten Services mit Kubernetes orchestriert. Kubernetes bringt eigene Konzepte für Deployment, Service Discovery und Health-Monitoring mit.



Namespace

Logische Trennung von Ressourcen im Cluster



Deployment

Beschreibt gewünschten Zustand der Pods



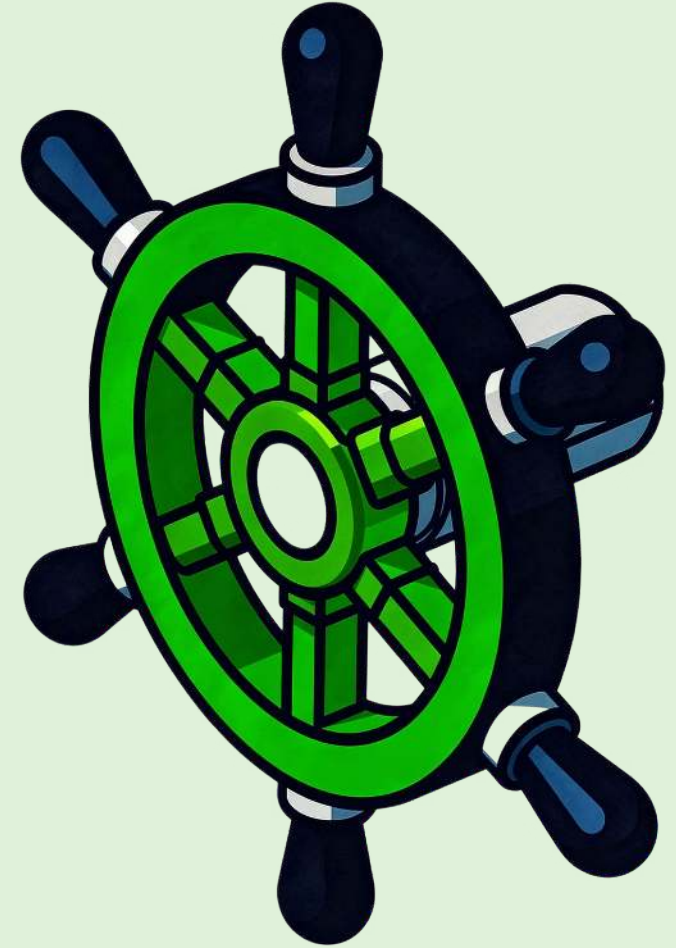
Service

Stabile interne Adresse für Pods



ConfigMap

Konfiguration außerhalb des Images verwalten



Kubernetes: Deployment

Ein Deployment beschreibt den **gewünschten Zustand** der Anwendung – Kubernetes sorgt dafür, dass dieser Zustand eingehalten wird.

Image-Konfiguration

Welches Docker-Image gestartet wird und mit welcher Pull-Policy

Replicas

Wie viele Instanzen (Pods) des Services gleichzeitig laufen sollen

Labels

Welche Labels die Pods bekommen, um von Services selektiert zu werden

Health Probes

Welche Readiness- und Liveness-Probes verwendet werden

 `imagePullPolicy: Never` wird für lokale Docker-Desktop-Images verwendet, damit Kubernetes keine Remote-Registry kontaktiert.

Kubernetes: Service Discovery

Ein Kubernetes **Service** ist die stabile interne Adresse für eine Gruppe von Pods. Pods können ersetzt werden, ohne dass andere Services ihre URL ändern müssen.

product-service

http://product-service:3001

inventory-service

http://inventory-service:3002

order-service

http://order-service:3003

- ✅ Die Service-URLs bleiben konstant, egal wie oft die dahinterliegenden Pods neu gestartet oder ersetzt werden – das ist der Kern von Service Discovery in Kubernetes.

Kubernetes: Probes

Kubernetes nutzt eigene Health-Konzepte, die die Compose-Healthchecks in der Kubernetes-Welt ersetzen:

readinessProbe

Der Pod bekommt erst dann Traffic zugewiesen, wenn er bereit ist. Solange die Probe fehlschlägt, wird der Pod aus dem Load-Balancer entfernt.

livenessProbe

Der Pod wird automatisch neu gestartet, wenn er nicht mehr gesund ist. Kubernetes erkennt so hängende oder abgestürzte Prozesse und reagiert selbstständig.

- ❗ Diese beiden Probe-Typen ersetzen die Compose-Healthchecks vollständig in der Kubernetes-Welt und bieten darüber hinaus automatisches Selbstheilungsverhalten.

GitHub Actions

In der sechsten Iteration wird die **automatisierte Prüfung bei jedem Push und Pull Request** eingerichtet. Gleiche Schritte werden für alle Services ausgeführt.

1

Checkout

Repository-Code auschecken

2

Node.js einrichten

Laufzeitumgebung konfigurieren

3

Dependencies installieren

npm-Pakete installieren

4

Tests ausführen

Automatisierte Tests laufen lassen

5

Docker Image bauen

Reproduzierbares Image erstellen

Matrix Build

Die CI nutzt eine **Matrix-Strategie**, um denselben Workflow für jeden Service auszuführen – ohne Wiederholung.

product-service

Produktdaten-Service wird automatisch gebaut und getestet

inventory-service

Lagerbestand-Service wird automatisch gebaut und getestet

order-service

Bestell-Service wird automatisch gebaut und getestet

api-gateway

Gateway wird automatisch gebaut und getestet

✅ Das reduziert Wiederholung im Workflow-Code und macht neue Services leichter integrierbar – einfach zur Matrix hinzufügen.

OPTIONAL

Image Publishing

`docker-build.yml` kann Docker-Images nach **GHCR (GitHub Container Registry)** pushen – das ist der nächste Schritt von CI zu CD.



docker/login-action

Authentifizierung bei der Container-Registry mit GitHub Token



docker/build-push-action

Image bauen und direkt in die Registry pushen



packages: write

GitHub Token benötigt die Berechtigung, Pakete in die Registry zu schreiben

i Mit Image Publishing in GHCR können die gebauten Images direkt in Kubernetes-Deployments referenziert werden – der Übergang von CI zu vollständigem CD.

Vergleich der Stufen

Jede Iteration baut auf der vorherigen auf und führt neue Konzepte ein:

Stufe	Fokus
Lokal	APIs und Prozesse – Services als eigenständige Node.js-Prozesse verstehen
Dockerfiles	Images und Container – Services in isolierte Container verpacken
Compose	Gemeinsamer lokaler Betrieb – alle Services mit einem Befehl starten
Healthchecks	Betriebsbereitschaft – Unterschied zwischen laufendem Container und bereitem Service
Kubernetes	Orchestrierung – Deployments, Service Discovery und Probes in der Praxis
GitHub Actions	Automatisierung – CI-Pipeline für alle Services mit Matrix-Build

Typische Fehlerbilder

Diese Fehler sind **Teil des Lernziels** – wer sie einmal erlebt hat, versteht die Konzepte dahinter viel besser.

Falsche Service-URL

Der Service wird unter einer falschen Adresse angesprochen – z.B. `localhost` statt Container-Name

Port-Konfigurationsfehler

Port lokal erreichbar, aber intern falsch konfiguriert – Verwechslung von Host-Port und Container-Port

Container läuft, Service nicht bereit

Der Container ist gestartet, aber der Service hat noch nicht vollständig initialisiert

Kubernetes findet lokales Image nicht

Fehlende `imagePullPolicy: Never` führt dazu, dass Kubernetes versucht, das Image remote zu laden

Environment Variable fehlt

Eine fehlende Umgebungsvariable führt zu Verbindungsfehlern zwischen Services

Best Practices im Projekt

Das Projekt demonstriert bewährte Praktiken für den Betrieb von Microservices:



Ein Service pro Container

Klare Trennung von Verantwortlichkeiten



Environment Variables

Konfiguration außerhalb des Codes verwalten



Health-Endpunkte

Jeder Service stellt einen `/health`-Endpunkt bereit



npm ci

Reproduzierbare Installation mit exakten Versionen



Infrastruktur als Code

Alle Konfigurationen sind versioniert und nachvollziehbar



Kleine Iterationen

Jede Stufe baut auf der vorherigen auf und ist für sich verständlich

Fazit

Das Projekt zeigt den Weg von einfachen lokalen APIs zu einer kleinen Microservice-Landschaft – Schritt für Schritt, mit klarem Fokus auf die Konzepte.



CI schafft Wiederholbarkeit

GitHub Actions stellt sicher, dass jeder Build reproduzierbar ist



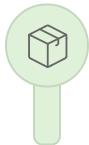
Kubernetes formalisiert Betriebskonzepte

Deployments, Probes und Services bringen Struktur in den Betrieb



Compose vereinfacht lokalen Betrieb

Alle Services mit einem Befehl starten und konfigurieren



Docker löst Packaging

Jeder Service wird als reproduzierbares Image verpackt

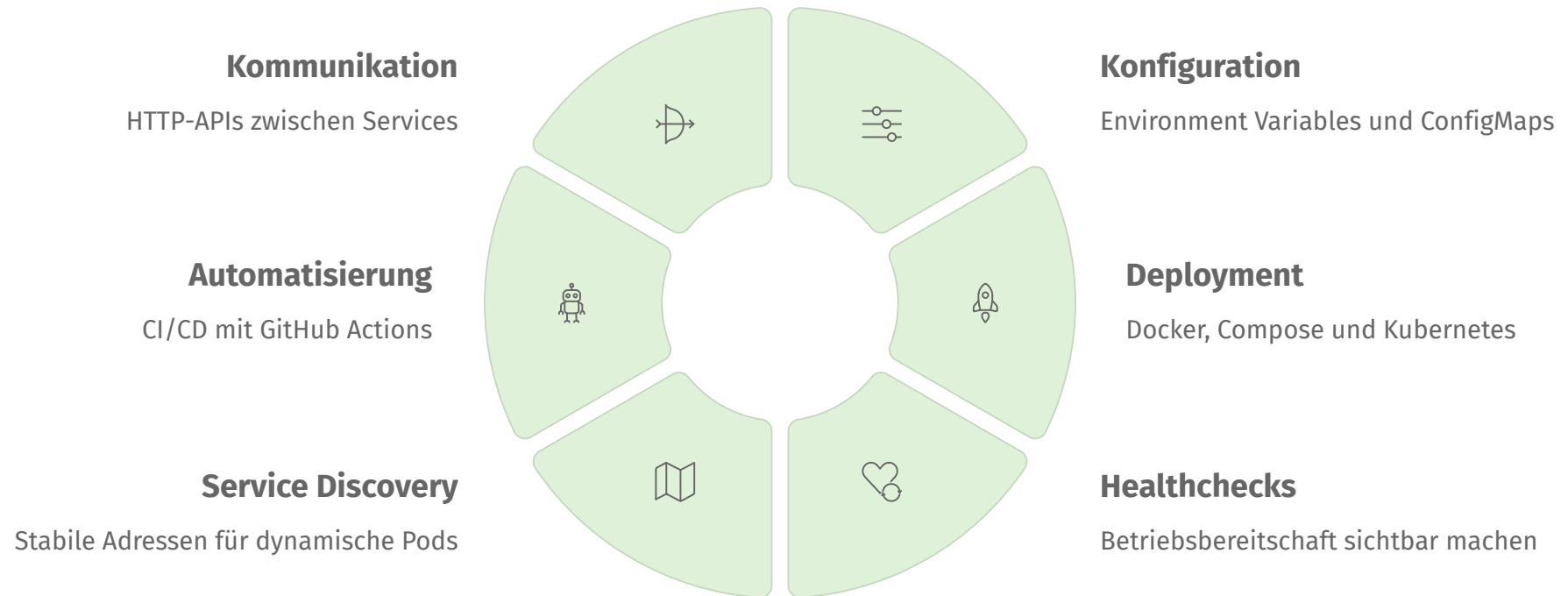


Services hängen voneinander ab

Abhängigkeiten werden sichtbar und müssen bewusst verwaltet werden

Zusammenfassung

Microservices sind nicht nur Code-Struktur – sie betreffen Kommunikation, Konfiguration, Deployment, Healthchecks, Service Discovery und Automatisierung.



Der Mini-Shop ist bewusst klein gehalten, damit diese Konzepte **klar erkennbar bleiben**.